○ E-LEARNING · STUDY NOTES

Nicloud · Kubernetes on AWS

Module 4 – Observability, GitOps and backup

Contents

Learning objectives 2
Architecture 2
kube-prometheus-stack 2
Example: Prometheus in this repo 3
Alerts and routing 4
Grafana and credentials outside Git 5
VictoriaMetrics 5
Logs with Fluent Bit 6
Uptime Kuma: the outside view 7
ArgoCD and GitOps 7
Velero: backup and restore 8
Summary 10
Quiz 10
Additional resources 11

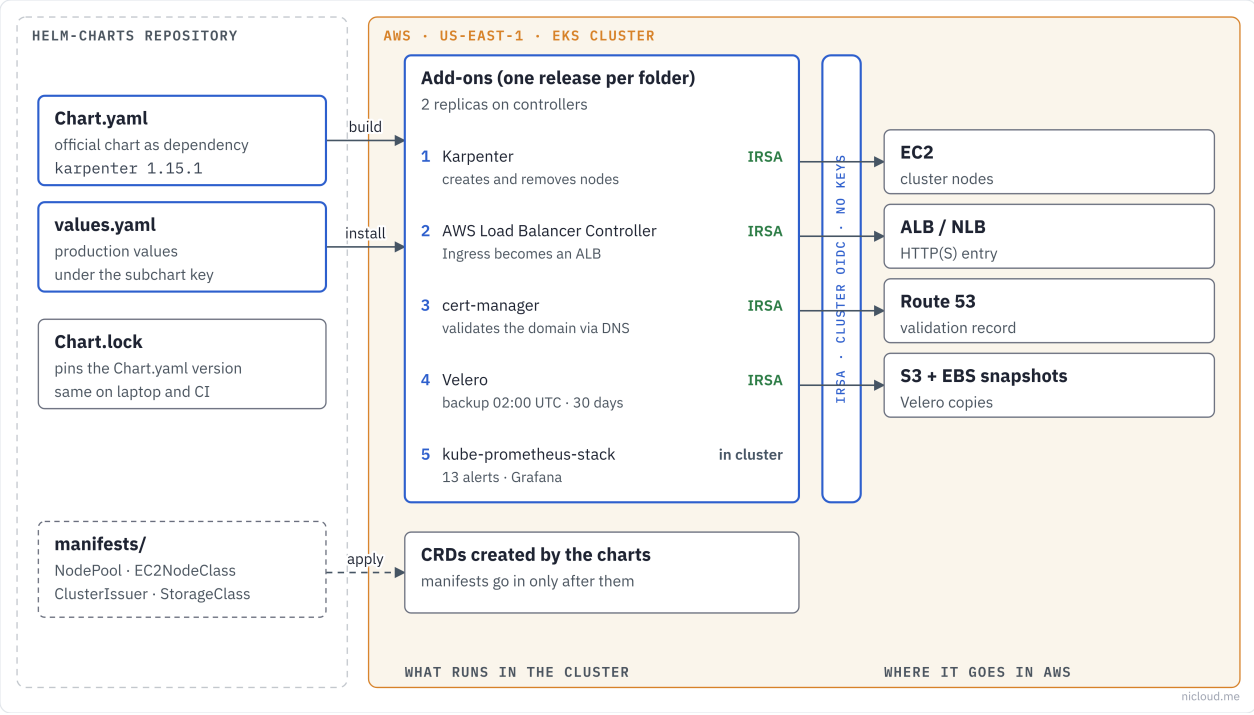
Learning objectives

In this module, you will learn to:

- Describe the parts of **kube-prometheus-stack**: Prometheus, the **Prometheus Operator**, Alertmanager and Grafana.
- Read an alert rule (`expr`, `for`, `severity`) and the **Alertmanager** routing tree.
- Compare Prometheus with **VictoriaMetrics** on retention, disk and cost.
- Ship pod logs to **CloudWatch Logs** with **Fluent Bit**.
- Monitor the system from the outside, the way users see it, with **Uptime Kuma**.
- Manage the addons through **GitOps** with **ArgoCD**.
- Configure cluster backup and restore with **Velero** and say what it does **not** cover.

Architecture

The diagram shows the overview. This module covers what watches the cluster (metrics, logs, external checks), what keeps the cluster matching Git (ArgoCD) and what brings the cluster back if it is lost (Velero).



kube-prometheus-stack

The `prometheus-stack` chart installs **kube-prometheus-stack**, a bundle with everything a cluster needs for metrics:

Part	Role
Prometheus	scrapes and stores metrics (time series)
Prometheus Operator	creates and configures Prometheus from CRDs
Alertmanager	groups alerts and sends them to Slack, PagerDuty, e-mail
Grafana	dashboards
node-exporter	per-node metrics (CPU, memory, disk)
kube-state-metrics	metrics about Kubernetes objects (pods, deployments, PVCs)

The **Prometheus Operator** is what makes the bundle easy to extend. Instead of editing a Prometheus configuration file, each team creates Kubernetes resources:

- **ServiceMonitor**: "scrape the metrics of this Service". The Kong chart from module 2 creates its own with `serviceMonitor.enabled: true`.
- **PrometheusRule**: alert rules.
- **AlertmanagerConfig**: alert routing for a namespace.

Example: Prometheus in this repo

```
# prometheus-stack/values.yaml (excerpt)
kube-prometheus-stack:
  defaultRules:
    create: true
  prometheus:
    prometheusSpec:
      retention: 15d
      resources:
        requests: { cpu: 500m, memory: 1Gi }
        limits:   { cpu: "1",  memory: 2Gi }
      storageSpec:
        volumeClaimTemplate:
          spec:
            storageClassName: gp3
            resources:
              requests:
                storage: 50Gi
```

- `retention: 15d`: Prometheus keeps 15 days. More than that needs a bigger disk or long-term storage (VictoriaMetrics, Amazon Managed Prometheus).
- `storageSpec` with 50 GiB of **gp3**: without a volume, the metrics die with the pod.
- `defaultRules.create: true` turns on the project's dozens of ready-made rules (etcd, API server, nodes, pods).
- The 2 GiB memory limit is the first to break as the number of series grows. If Prometheus restarts with **OOMKilled**, the problem is **cardinality** (labels with many values, such as a user ID), not a lack of memory.

Alerts and routing

Besides the default rules, the repository has 13 rules of its own in `prometheus-stack/manifests/alert-rules.yaml`. A rule has three parts:

```
# prometheus-stack/manifests/alert-rules.yaml (excerpt)
- alert: PodCrashLooping
  expr: increase(kube_pod_container_status_restarts_total[15m]) > 5
  for: 0m
```

- `expr`: the **PromQL** query. Here, "the container restarted more than 5 times in the last 15 minutes".
- `for`: how long the condition must stay true before firing. With `5m`, a two-minute spike wakes nobody up.
- `labels.severity`: decides where the alert goes.

The 13 rules: `NodeNotReady`, `ClusterDown`, `HighErrorRate`, `HighLatency`, `PodCrashLooping`, `PodPending`, `DeploymentReplicasMismatch`, `PVCAlmostFull`, `DiskUsageHigh`, `HighCPU`, `HighMemory`, `CertificateExpiring30Days` and `CertificateExpiringSoon`.

Alertmanager receives the alerts and decides what to do with them:

```
# prometheus-stack/manifests/alertmanager-config.yaml (excerpt)
route:
  receiver: slack-notifications
  groupBy: ["alertname", "namespace", "pod"]
  groupWait: 30s
  groupInterval: 5m
  repeatInterval: 4h
  routes:
    - receiver: pagerduty-critical
      matchers:
        - { name: severity, value: critical, matchType: "=" }
      repeatInterval: 15m
      continue: true
```

- `groupBy`: alerts with the same name, namespace and pod become **one** notification. Without it, a failing node sends one message per pod.
- `groupWait: 30s`: waits 30 seconds to gather alerts of the same group before the first message.
- `repeatInterval: 4h`: if it keeps firing, it reminds you every 4 hours. Critical ones remind every 15 minutes.
- `continue: true`: the critical alert goes to PagerDuty **and keeps going** down the tree, reaching Slack too.

Grafana and credentials outside Git

```
# prometheus-stack/values.yaml (excerpt)
grafana:
  enabled: true
  admin:
    existingSecret: grafana-admin
    userKey: admin-user
    passwordKey: admin-password
  service:
    type: ClusterIP
  persistence:
    enabled: true
    size: 10Gi
    storageClassName: gp3
```

- `admin.existingSecret`: the admin password comes from a Secret created outside Git (`kubectl create secret generic grafana-admin …`, or by the CSI driver from module 3). No password in `values.yaml`.
- `service.type: ClusterIP`: Grafana is not exposed to the Internet. Reach it with `kubectl port-forward` or put it behind an authenticated Ingress.

The Slack webhook follows the same rule. Instead of `api_url` with the URL in the file, Alertmanager uses `api_url_file`, and the file comes from a Secret mounted through `alertmanagerSpec.secrets`:

```
# prometheus-stack/values.yaml (excerpt)
alertmanager:
  alertmanagerSpec:
    secrets:
      - alertmanager-slack
  config:
    receivers:
      - name: slack
        slack_configs:
          - api_url_file: /etc/alertmanager/secrets/alertmanager-slack/webhook-
url
```

Until November 2026, this file had `adminPassword: "CHANGE_ME"`, Grafana on a public LoadBalancer and the webhook in plain text. All three were fixed together with the version upgrade.

Dashboards can come from ConfigMaps. `grafana-dashboards.yaml` carries the `grafana_dashboard: "1"` label, and Grafana's **sidecar** imports the dashboard on its own.

VictoriaMetrics

VictoriaMetrics speaks the same language as Prometheus (same scraping, PromQL queries, same rule format), but stores metrics with much less disk and memory. It is the option when 15 days of retention are not enough.

```
# victoria-metrics/values.yaml (excerpt)
victoria-metrics-k8s-stack:
  vmsingle:
    enabled: true
    spec:
      retentionPeriod: "90d"
      storage:
        storageClassName: gp3
        resources:
          requests:
            storage: 100Gi
  vmagent:  { enabled: true }
  vmalert:  { enabled: true }
  grafana:  { enabled: false }
```

- `vmsingle`: a single node with 90 days of retention on 100 GiB. This repository's Prometheus keeps 15 days on 50 GiB.
- `vmagent` scrapes, and `vmalert` evaluates rules.
- `grafana.enabled: false`: the Grafana from `prometheus-stack` already exists. Just add VictoriaMetrics to it as a data source.

Both charts are in the repository as alternatives. A cluster usually runs one or the other, or Prometheus for short-term alerting and VictoriaMetrics for history.

Logs with Fluent Bit

Fluent Bit runs as a **DaemonSet** (one pod per node), reads the log files of every container on the node and sends them to a destination. Here, the AWS chart sends them to **CloudWatch Logs**:

```
# logging/values.yaml (excerpt)
aws-for-fluent-bit:
  cloudWatch:
    enabled: true
    region: "us-east-1"
    logGroupName: "/eks/CLUSTER_NAME"
    autoCreateGroup: false
  firehose: { enabled: false }
  kinesis:  { enabled: false }
  serviceAccount:
    annotations:
      eks.amazonaws.com/role-arn: "CHANGE_ME"
  resources:
    requests: { cpu: 100m, memory: 128Mi }
    limits:   { memory: 512Mi }
```

- `autoCreateGroup: false`: the log group must exist beforehand, created by Terraform, with a **retention period**. An automatically created group has infinite retention, and the storage bill only grows.
- The IRSA role needs `logs:PutLogEvents` and `logs:CreateLogStream` on that group.

- There is a memory limit and no CPU limit. If Fluent Bit is killed for memory, that node's logs stop arriving, and it is better to let it use CPU during spikes than to lose logs.
- The `livenessProbe` on port 2020 restarts the pod if it hangs.

Uptime Kuma: the outside view

Internal metrics say the pods are healthy. They do not say whether users can open the site. **Uptime Kuma** does **synthetic monitoring**: it hits the URLs periodically, from the outside, and alerts when they stop answering.

```
# uptime-kuma/values.yaml (excerpt)
uptime-kuma:
  volume:
    enabled: true
    size: 5Gi
    storageClassName: gp3
  ingress:
    enabled: true
    annotations:
      cert-manager.io/cluster-issuer: letsencrypt-production
    hosts:
      - host: status.example.com
```

This addon brings three modules together: the Ingress (module 2) publishes the page, **ExternalDNS** creates `status.example.com` and **cert-manager** issues the certificate through the `letsencrypt-production` ClusterIssuer. The issuer name was wrong (`letsencrypt-prod`) until November 2026.

The weakness of a monitor inside the cluster is obvious: if the cluster goes down, the monitor goes with it. For the most important "it's down" alert, also use an external service.

ArgoCD and GitOps

GitOps means running the cluster from Git: the desired state lives in the repository, and an agent inside the cluster compares what is running with Git and fixes the difference. Nobody runs `helm upgrade` by hand.

ArgoCD is that agent:

```
# argocd/values.yaml (excerpt)
argo-cd:
  server:
    replicas: 2
    service:
      type: ClusterIP
  controller:
    replicas: 1
  repoServer:
    replicas: 2
```

- `server` (API and UI) and `repoServer` (which fetches Git and renders the manifests, including running Helm) have two replicas each.
- The `controller` has **one**. More controller replicas **split clusters** among themselves (sharding); they do not create standby copies for the same cluster. With a single cluster, one replica is enough. If it fails, Kubernetes recreates the pod, and nothing already running stops.

Each folder in this repository becomes an **Application**:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: ingress-nginx
  namespace: argocd
spec:
  source:
    repoURL: <repository URL>
    targetRevision: main
    path: ingress-nginx
  destination:
    server: https://kubernetes.default.svc
    namespace: ingress-nginx
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

- `automated.selfHeal`: if someone changes something by hand in the cluster, ArgoCD reverts it.
- `automated.prune`: what leaves Git leaves the cluster.
- Folders with `manifests/` need an Application with **two sources** (the chart and the manifests folder) or a second Application, applied afterwards, following the order from module 1.

With many Applications, the **app of apps** pattern helps: a root Application points at a folder with the others, and the whole cluster comes up from a single resource.

Velero: backup and restore

Velero backs up the **Kubernetes resources** (the YAML of everything in the cluster) to S3 and the **volumes** as EBS snapshots.

```
# velero/values.yaml (excerpt)
velero:
  initContainers:
    - name: velero-plugin-for-aws
      image: velero/velero-plugin-for-aws:v1.14.4
  configuration:
    backupStorageLocation:
      - name: default
        provider: aws
        bucket: "CHANGE_ME"
    volumeSnapshotLocation:
      - name: default
        provider: aws
  schedules:
    daily-backup:
      schedule: "0 2 * * *"
      template:
        ttl: "720h"
```

- The **AWS plugin** comes in as an init container and teaches Velero to talk to S3 and EBS. The plugin version must match the Velero version: chart 12.2.3 ships Velero 1.18, which needs plugin 1.14.
- `backupStorageLocation`: the bucket for the YAML. Use a bucket with versioning and, ideally, in another account, so an attacker in the cluster cannot delete the backups.
- `volumeSnapshotLocation`: where the EBS snapshots go.
- `schedule: "0 2 * * *"` with `ttl: 720h`: daily backup at 02:00 UTC, kept for 30 days.

How it works

(1) Backup

At the scheduled time, Velero lists the resources through the API, writes them as `.tar.gz` to S3 and asks AWS for a snapshot of each EBS volume.

(2) Restore

`velero restore create --from-backup` <name> recreates the resources and creates new volumes from the snapshots. You can restore a single namespace (`--include-namespaces`) or only some types (`--include-resources`).

What Velero does not cover

The `disaster-recovery.md` guide lists what stays outside Velero, because it is an AWS service and not a cluster resource:

Component	How to protect it
RDS	automated and manual snapshots
S3 data	versioning and cross-region replication
Route 53, IAM, VPC	Terraform (the code is the backup)
EKS control plane	managed by AWS, recreated by Terraform

With daily backups, the **RPO** (how much data you accept losing) for Velero-backed scenarios is up to **24 hours**. If that is not enough, increase the frequency. The guide also gives the **RTO** (how long you are down) for each scenario: under a minute for a pod, up to 4 hours for a whole region.

A backup that has never been restored is a hypothesis. Restore a namespace into a test cluster at least once a quarter.

Summary

kube-prometheus-stack brings Prometheus, Alertmanager, Grafana and the exporters, and the **Prometheus Operator** lets each team add scrapes and alerts with CRDs. Here Prometheus keeps 15 days on 50 GiB of gp3, and the 13 custom rules go through **Alertmanager**, which groups them, sends critical ones to PagerDuty and Slack and reminds every 15 minutes or 4 hours. The Grafana password and the Slack webhook come from Secrets outside Git, and Grafana is not exposed to the Internet.

When 15 days are not enough, **VictoriaMetrics** keeps 90 days with fewer resources and the same queries. Logs leave each node through **Fluent Bit** for a CloudWatch log group with a set retention, and **Uptime Kuma** checks from the outside whether users can get in, using the Ingress, ExternalDNS and cert-manager from module 2.

ArgoCD keeps the cluster matching Git, with one Application per folder, self-healing and removal of what left the repository. **Velero** backs up resources and volumes daily, kept for 30 days. It does not cover RDS, S3 or AWS infrastructure, and it is only trustworthy after you have restored from it at least once.

Quiz

A team wants Prometheus to scrape its service's metrics. What do they create, without touching the Prometheus configuration?

- A **ServiceMonitor** pointing at the Service. The Prometheus Operator reads the resource and configures the scrape.

The `HighCPU` rule has `for: 5m`. A node's CPU stayed at 95% for 3 minutes and went back to normal. Did the alert fire?

- No. The condition had to stay true for 5 minutes in a row.

A node failed with 40 pods on it. Why did Slack get a few messages, not 40?

- Alertmanager's `groupBy` and `groupWait` merge similar alerts into one notification.

A `critical` alert reached PagerDuty and also Slack. Which setting did that?

- `continue: true` on the PagerDuty route. Without it, the alert would stop at the first matching route.

Prometheus restarts with OOMKilled every afternoon after a new deploy of an application. What is the first suspect?

- Cardinality: the new application exposes metrics with high-variety labels (user ID, full URL). Adding memory only postpones the problem.

You need 90 days of metrics for capacity planning. What do you use in this repository?

- VictoriaMetrics (`retentionPeriod: 90d`), as a data source in the same Grafana.

The CloudWatch Logs bill grows every month even though log volume is stable. What do you check in Fluent Bit?

- Whether the log group has a retention period. With `autoCreateGroup: true`, the group would be created with infinite retention. Here it should be created by Terraform with a set retention.

Someone edited the replica count of an ArgoCD-managed deployment by hand. What happens?

- With `selfHeal: true`, ArgoCD reverts it to the value in Git.

ArgoCD manages a single cluster. Is it worth raising the `controller` to 3 replicas?

- No. Controller replicas split clusters among themselves (sharding). With one cluster, one replica does all the work.

The cluster was lost at 18:00, and the last Velero backup is from 02:00. How much cluster data may have been lost, and what does not come back through Velero?

- Up to 16 hours of changes to resources and volumes. RDS, S3 data and AWS resources such as IAM and Route 53 come back through other paths (snapshots, versioning, Terraform).

Additional resources

- kube-prometheus-stack
- Prometheus Operator: getting started
- Alertmanager: configuration
- VictoriaMetrics: Kubernetes stack
- Fluent Bit on EKS (AWS)
- Uptime Kuma
- ArgoCD: core concepts
- Velero: backup and restore
- Velero plugin for AWS